PyTorch 2.15 Cheat Sheet

Python 3.13, PyTorch 2.15 (released 2026-10-28), checked October 2026

Install and index URLs

```
python3.13 -m venv .venv && source
.venv/bin/activate
pip install torch torchvision \
  --index-url
https://download.pytorch.org/whl/cu132
# whl/cpu | whl/rocm7.14 | whl/xpu |
whl/nightly/<p> (--pre)
```

- 2.15 needs Python 3.11+; stable CUDA 13.2, ROCm 7.14
- Wheels bundle CUDA; only a driver is needed
- Official conda channel ended after 2.5
- One index per command; install torch* together

Tensors and devices

```
x = torch.randn(2, 3, dtype=torch.float32)
dev =
torch.accelerator.current_accelerator()
x = x.to(dev)            # or
torch.device("cpu")
torch.accelerator.is_available()
x.view(3, 2); x.reshape(-1);
x.T.contiguous()
```

- `view` shares memory; `reshape` may copy
- `from_numpy` shares memory with the array
- Keep tensors on one device; `pin_memory` + `non_blocking=True`

Autograd and grad modes

`requires_grad=True`	track operations
`loss.backward()`	accumulates into `.grad`
`no_grad()`	no graph; evaluation
`inference_mode()`	fastest; no autograd later
`detach()`	cut the graph
`torch.func.grad`	functional, higher-order

nn.Module skeleton

```
class Net(nn.Module):
    def __init__(self, d_in, d_out):
        super().__init__()
        self.f = nn.Sequential(
            nn.Linear(d_in, 64), nn.ReLU(),
            nn.Linear(64, d_out))
    def forward(self, x):
        return self.f(x)
model = Net(10, 3).to(dev)
model.train(); model.eval()
```

Buffers via `register_buffer`; params via `nn.Parameter`.

Losses and optimisers

Regression	`MSELoss`, `SmoothL1Loss`
Binary	`BCEWithLogitsLoss`
Multiclass	`CrossEntropyLoss` (raw logits, class indices)
AdamW	`AdamW(p, lr=1e-3, weight_decay=0.01)`
SGD	`SGD(p, lr=0.1, momentum=0.9)`
Schedules	`CosineAnnealingLR`, `OneCycleLR`, `LinearLR`

Canonical training loop

```
for epoch in range(E):
    model.train()
    for x, y in train_dl:
        x, y = x.to(dev), y.to(dev)
        opt.zero_grad(set_to_none=True)
        loss = crit(model(x), y)
        loss.backward()
        clip_grad_norm_(model.parameters(),
1.0)
        opt.step()
    sched.step()
    model.eval()
    with torch.no_grad():
        ...  # validation
```

DataLoader knobs

`batch_size`	per step; `drop_last` for fixed shapes
`shuffle=True`	train only
`num_workers`	4-8 per GPU; 0 to debug
`pin_memory=True`	faster host-to-GPU copies
`persistent_workers`	keep workers between epochs
`prefetch_factor`	batches per worker (default 2)
`DistributedSampler`	call `set_epoch` each epoch

Save and load (weights_only)

```
torch.save(model.state_dict(), "m.pt")
sd = torch.load("m.pt", map_location="cpu",
                weights_only=True)   #
default since 2.6
model.load_state_dict(sd)
```

- Checkpoint: model, optimiser, scheduler, epoch
- `weights_only=False` only for trusted files
- Allow custom classes with `add_safe_globals`

AMP and torch.compile

```
with torch.autocast("cuda",
dtype=torch.bfloat16):
    loss = crit(model(x), y)
scaler = torch.amp.GradScaler("cuda")   #
fp16 only
model = torch.compile(model)
```

- bf16 needs no scaler; fp16 does
- First call compiles; avoid graph breaks
- `fullgraph=True` to find breaks

DDP and FSDP2 launch

```
torchrun --nproc_per_node=8 train.py
# DDP: init_process_group("nccl")
model = DDP(model.to(rank), device_ids=
[rank])
# FSDP2: shard each block, then the root
for b in model.blocks: fully_shard(b)
fully_shard(model)
```

- DDP: replicas, all-reduce of gradients; model fits on one GPU
- FSDP2 (`fully_shard`): shards params, grads, state
- Save with Distributed Checkpoint (DCP)

Export paths

`torch.export.export`	`ExportedProgram`, full graph, `.pt2`
AOTInductor	compiled `.pt2` for C++ serving
ONNX	`torch.onnx.export` (`dynamo=True` default since 2.9) to ONNX Runtime
ExecuTorch	`.pte` for phones and embedded

Export in `eval()` mode with example inputs and `dynamic_shapes`.

Legacy to current

`Variable`, `volatile`	plain tensors, `no_grad`
`torch.jit.script/trace`	`torch.export`, `torch.compile`
`torch.cuda.amp`	`torch.amp.autocast`, `GradScaler("cuda")`
`torch.ao.quantization`	`torchao`
FSDP1 wrapper	`fully_shard`
TorchServe, torchtext	archived; use other stacks
pickle `load` default	`weights_only=True`
conda `-c pytorch`	pip wheels

Generated with AI assistance; verify against official documentation. Full detail in the PyTorch part of the docs site.